Examen Programmation C++ - Année 2021/2022

Durée : 1h20 - Documents papiers autorisés

Exercice 1 :

On se propose de créer une classe *Secret* permettant de stocker un code secret sous la forme d'une chaine de caractères. Ce code secret pourra être constitué de 4 chiffres (par ex: "1337") ou 6 chiffres (par ex. "314159") avec une valeur par défaut de 4 chiffres égale à "0000".

Q1 Ecrire la déclaration de la classe *Secret* et le code du ou des constructeurs permettant d'écrire les instructions suivantes :

```
Secret s1;                    // code avec "0000" pour valeur
Secret s2("1337");            // code avec "1237" pour valeur
Secret s3("314159");          // code avec "314159" pour valeur
```

Q2 Écrire le code de la méthode nommée *longueur* permettant de déterminer la taille du code stocké

```
int l1 = longueur(s1);      // renvoie 4
int l3 = longueur(s3);      // renvoie 6
```

Q3 Écrire le code de la méthode nommée *compare* permettant de comparer la valeur du code stocké avec un code passé en paramètre

```
bool b2 = s2.compare("2001");          // b2 = False
bool b3 = s3.compare("314159");        // b3 = True
```

Q4 Ecrire la surcharge de l'opérateur == permettant de comparer la valeur du code stocké avec un autre code (la surcharge de == pourra remplacer la fonction *compare* de la question Q4)

```
bool b2bis = (s2 == "2001");              // b2bis = False
bool b3bis = (s3 == "314159");            // b3bis = True
```

Q5 Ecrire le code de la fonction *verifie* permettant de vérifier qu'un code est correct (nombre positif de 4 chiffres ou 6 chiffres). On pourra utiliser std::isDigit(char c) qui renvoie True si le caractère « *c* » est effectivement un chiffre compris entre 0 et 9.

```
verifie("1234");              // True
verifie ("123");              // False car 3 caractères seulement
verifie ("-123");             // False car signe -
verifie ("12a3");             // False car lettre a
verifie ("123456789");        // False car 9 caractères
```

Exercice 2 :

On se propose de définir une classe C++ nommée *Vecteur* qui contient une seule variable nommée *data* de type std ::vector

Q1 Ecrire la déclaration de la classe *Vecteur* et le code du ou des constructeurs permettant de créer soit un Vecteur vide, soit un Vecteur initialisé avec des données passées en paramètres comme dans le code ci-dessous.

```
Vecteur v1; // Vecteur vide
std::vector<int> datatest = {-1,3,-5,8,4};
Vecteur v2(datatest); // Vecteur initialisé avec un ensemble de données
```

Q2 On se propose de pouvoir ajouter une nouvelle donnée au Vecteur. Ecrire le code de la méthode *add* permettant d'exécuter le code suivant :

```
v2.add(-6); v2.add(10);
```

Q3 On se propose de pouvoir afficher le contenu de la variable data d'un vecteur. Ecrire la surcharge de l'opérateur << permettant d'exécuter le code suivant :

```
std::cout << "v1 : " << v1;     // Affiche v1 :
std::cout << "v2 : " << v2;     // Affiche v2 : -1 3 -5 8 4 -6 10
```

Q4 On se propose de pouvoir extraire uniquement les nombres positifs contenus dans le vecteur avec une méthode ou une fonction helper. Attention, la méthode/fonction nommée *positive* devra créer un nouvel objet de type Vecteur et ne pas modifier le Vecteur original. Ecrire le code de la méthode ou de la fonction helper pouvant être exécutée de la manière suivante :

```
// Exemple d'utilisation de la méthode
Vecteur v3 = v2.positive();
std::cout << "v2 > 0 : " << v3;   // Affiche 3 8 4 10

// Exemple d'utilisation de la fonction helper
Vecteur v3bis = positive(v2) ;
std::cout << "v2 > 0 : " << v3bis;      // Affiche 3 8 4 10
```

Q5 On se propose de trier les nombres du vecteur avec une méthode/fonction helper. Ecrire le code de la méthode ou de la fonction helper nommée *sort* pouvant être exécutée de la manière suivante :

```
// Exemple d'utilisation de la méthode
Vecteur v4 = v2.sort();
std::cout << "tri v2 : " << v4;         // Affiche -6 -5 -1 3 4 8 10

// Exemple d'utilisation de la fonction helper
Vecteur v4bis = sort(v2) ;
std::cout << "tri v2 : " << v4bis;      // Affiche -6 -5 -1 3 4 8 10
```